

claude-windows / 7fe4ef45-d30a-448e-98eb-4da4fba244f1

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fe4ef45-d30a-448e-98eb-4da4fba244f1\subagents\workflows\wf\_b531eaeef-39c\agent-af4e5de58395aa35c.jsonl`
- SHA-256: `cc0afdbd5d34d9acb993902328d1599b8c30e60f4886f8cdbf2ed5fb84f5deea`
- Source modified: `2026-06-19T16:57:54+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf\_b531eaeef-39c`
- session_id: `7fe4ef45-d30a-448e-98eb-4da4fba244f1`

Transcript

1. user (2026-06-19T16:57:07.030Z)

You are verifying claims for a design doc; the doc must only assert what the code supports. Verify: backend/pipeline/detectors/court_gate.py's gate_intervals_to_court is a WINDOW-LEVEL, SUBTRACTIVE-ONLY post-filter operating on the single tracked trajectory (it can only DROP rallies, never recover one). Also verify point_in_polygon (backend/pipeline/detectors/fusion_features.py) and FusionConfig.court_polygon (backend/config/models.py) exist and are reusable as a track-SELECTION term. Where/how is the gate wired today (backend/eval/rally_seg_eval.py)? Cite lines. VERIFIED/REFUTED/PARTIAL per claim.

Read the actual files (do not guess). Return the structured verdict.

2. assistant (2026-06-19T16:57:09.275Z)

I'll verify these claims by reading the actual source files.

3. user (2026-06-19T16:57:10.247Z)

```
1      """Prominent-court shuttle gate ? multicourt G2 (see
2      ``docs/PROMINENT_COURT_DETECTION.md``).
3      In multicourt footage the WASB shuttle tracker follows ANY shuttle in frame, so a rally
4      on a background / adjacent court becomes a false rally on the filmed match. This gate re-
5      defines the shuttle signal ``(x, y, visible)`` ? ``(x, y, visible AND inside-the-prominent-court)``:
6      a visible shuttle point whose normalised position is outside the prominent-court polygon is marked
7      NOT visible, so the velocity windower never opens a rally there.
8
9      Pure (no GPU / cv2): point-in-polygon over the court polygon (reuses
10     ``fusion_features.point_in_polygon``). **Subtractive only** ? it can only CLEAR
11     visibility, never set it ? so on a correct polygon it suppresses out-of-court rallies without
12     manufacturing any.
13     ``polygon=None`` (or < 3 vertices, or non-positive frame dims) ? an EXACT no-op (single-
14     court / unconfigured footage), matching ``rally_gate``'s do-no-harm discipline.
15
16     The polygon is the court FLOOR in normalised 0-1 image coords (same convention as
17     ``FusionConfig.court_polygon``). ``height_pad`` is the MVP pseudo-3D play-volume ($4.3):
18     a point up to ``height_pad`` (normalised) ABOVE the floor ? i.e. higher in frame, smaller y ? is
19     still
```

```

18     admitted, so a high clear over the prominent court is not clipped.
19     """
20     from __future__ import annotations
21
22     from dataclasses import replace
23     from typing import Any, List, Optional, Sequence, Tuple
24
25     from backend.pipeline.detectors.fusion_features import point_in_polygon
26
27     Polygon = Sequence[Tuple[float, float]]
28
29
30     def admitted(nx: float, ny: float, polygon: Polygon, height_pad: float = 0.0) -> bool:
31         """Is the normalised point inside the court floor polygon, OR within ``height_pad``
above it?
32
33         The pseudo-3D air column: shifting the point DOWN by ``height_pad`` (toward the
floor) and
34         finding it inside means the original sits within ``height_pad`` above the floor ? a
high shuttle
35         over the prominent court, admit it."""
36         if point_in_polygon((nx, ny), polygon):
37             return True
38         return height_pad > 0.0 and point_in_polygon((nx, ny + height_pad), polygon)
39
40
41     def in_court_fraction(interval: Tuple[float, float], points: List[Any], polygon:
Polygon,
42                             fps: float, frame_width: float, frame_height: float,
43                             height_pad: float = 0.0) -> float:
44         """Fraction of a candidate rally's VISIBLE shuttle points that lie inside the
prominent court.
45
46         ``interval`` is ``(start_s, end_s)``; points carry ``.frame``. Returns 1.0 when
there are no
47         visible points in the interval (a window with no shuttle is not an out-of-court
false positive ?
48         leave that judgement to the windower)."""
49         if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0 or fps
<= 0:
50             return 1.0
51         fs, fe = interval[0] * fps, interval[1] * fps
52         n = inside = 0
53         for p in points:
54             if getattr(p, "visible", False) and fs <= p.frame <= fe:
55                 n += 1
56                 if admitted(p.x / frame_width, p.y / frame_height, polygon, height_pad):
57                     inside += 1
58         return 1.0 if n == 0 else inside / n
59
60
61     def gate_intervals_to_court(intervals: List[Tuple[float, float]], points: List[Any],
62                                polygon: Optional[Polygon], fps: float,
63                                frame_width: float, frame_height: float,
64                                min_in_court_frac: float = 0.5, height_pad: float = 0.0
65                                ) -> List[Tuple[float, float]]:
66         """Drop candidate rallies whose shuttle is MAJORITY out-of-court ? the window-level
gate.
67
68         Point-level gating (:func:`gate_points_to_court`) is leaky: a background-court rally
whose shuttle
69         occasionally drifts into the prominent court keeps a few in-court points, enough to
hold a (sparse)
70         window (measured: GX010142 rally #1 is 90% adjacent-court yet a 10% central tail
survives a hard

```

```

71         point gate). This drops an interval whose in-court point fraction is below
72         ``min_in_court_frac``,
73         so a rally that is *mostly* in the adjacent court is removed wholesale. No-op
74         (returns all intervals) when ``polygon`` is None/degenerate. Subtractive-only."""
75         if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0 or fps
76         <= 0:
77             return intervals
78         return [iv for iv in intervals
79                 if in_court_fraction(iv, points, polygon, fps, frame_width, frame_height,
80 height_pad)
81                 >= min_in_court_frac]
82
83     def gate_points_to_court(points: List[Any], polygon: Optional[Polygon],
84                             frame_width: float, frame_height: float,
85                             height_pad: float = 0.0) -> List[Any]:
86         """Return ``points`` with visibility CLEARED for any visible shuttle outside the
87         prominent-court
88         polygon (expanded upward by ``height_pad``).
89         No-op (returns the input list unchanged) when ``polygon`` is None/degenerate or the
90         frame dims
91         are non-positive. Otherwise each gated point is a NEW ``TrajectoryPoint`` (the input
92         list and
93         its points are never mutated)."""
94         if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0:
95             return points
96         out: List[Any] = []
97         for p in points:
98             if getattr(p, "visible", False):
99                 nx, ny = p.x / frame_width, p.y / frame_height
100                 if not admitted(nx, ny, polygon, height_pad):
101                     p = replace(p, visible=False)
102         out.append(p)
103         return out

```

4. user (2026-06-19T16:57:10.663Z)

```

1         """Pure per-window person-cue features for multi-signal rally fusion
2         (MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
3
4         NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
5         math is unit-testable in a GPU-free dev container. The detection (person boxes,
6         shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
7         module only turns those signals into a handful of **scale/translation-invariant**
8         features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
9         memorise this court/camera and fail to generalise off a small labelled set).
10
11         Conventions:
12         - A person box is ``(x1, y1, x2, y2)`` in **pixels** ; ``(track_id, box)`` when motion
13         needs it.
14         - ``per_frame`` is ``{frame_idx: [(track_id, box), ...]}`` keyed by the ORIGINAL video
15         frame
16         index (so it aligns directly with the shuttle trajectory's ``.frame`` ? no clip re-
17         timing).
18         - A shuttle point has ``.frame``, ``.visible``, ``.x``, ``.y`` (pixels) ? the WASB
19         trajectory shape.
20         - ``court_polygon`` is normalised 0-1 ``[(x, y), ...]`` or ``None`` (None ? no mask,
21         every
22         detection counts ? the player-count-only mode).
23         - ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided =
24         0.0).
25
26

```

```

19     Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20     floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
21     """
22     from __future__ import annotations
23
24     from typing import Dict, List, Optional, Sequence, Tuple
25
26     BBox = Tuple[float, float, float, float]
27     Net = Tuple[str, float]
28
29
30     def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31         """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32         (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33         if polygon is None or len(polygon) < 3:
34             return False
35         x, y = point
36         inside = False
37         n = len(polygon)
38         j = n - 1
39         for i in range(n):
40             xi, yi = polygon[i]
41             xj, yj = polygon[j]
42             # Does the horizontal ray at y cross edge (i, j)?
43             if (yi > y) != (yj > y):
44                 x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45                 if x < x_cross:
46                     inside = not inside
47             j = i
48         return inside
49
50
51     def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52         """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53         the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54         if frame_width <= 0 or frame_height <= 0:
55             return (0.0, 0.0)
56         x1, _y1, x2, y2 = box
57         return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60     def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61         if frame_width <= 0 or frame_height <= 0:
62             return (0.0, 0.0)
63         x1, y1, x2, y2 = box
64         return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67     def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                        court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69         """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70         True (count everyone) when it is None. Unusable frame dims (w/h <= 0) make the foot
71         point meaningless, so a masked check returns False rather than testing a bogus
72         (0,0)."""
73         if court_polygon is None:
74             return True
75         if frame_width <= 0 or frame_height <= 0:
76             return False
77         return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
78
79     def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,

```

```

80         frame_height: float,
81         court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
82     """Per-frame count of in-court persons (one entry per sampled frame)."""
83     return [
84         sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
court_polygon))
85         for _frame_idx, dets in sorted(per_frame.items())
86     ]
87
88
89     def _median(vals: Sequence[float]) -> float:
90         if not vals:
91             return 0.0
92         s = sorted(vals)
93         n = len(s)
94         return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
95
96
97     def players_in_court(counts: Sequence[int]) -> float:
98         """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
99         return _median(list(counts))
100
101
102     def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
103         """Fraction of sampled frames with >= `min_players` in court (track stability vs
flicker)."""
104         if not counts:
105             return 0.0
106         return sum(1 for c in counts if c >= min_players) / len(counts)
107
108
109     def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
110                           frame_width: float, frame_height: float) -> float:
111         """Mean per-track centre displacement between consecutive sampled frames, with x
112         normalised by frame width and y by frame height (per-axis; uniform-scale-invariant,
113         not aspect-ratio-isotropic). 0.0 if <2 frames or no shared tracks."""
114         if frame_width <= 0 or len(per_frame) < 2:
115             return 0.0
116         frames = sorted(per_frame.keys())
117         steps: List[float] = []
118         for a, b in zip(frames, frames[1:]):
119             prev = {tid: box for tid, box in per_frame[a]}
120             for tid, box in per_frame[b]:
121                 if tid in prev:
122                     cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
123                     cx1, cy1 = _center_norm(box, frame_width, frame_height)
124                     steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
125         if not steps:
126             return 0.0
127         return sum(steps) / len(steps)
128
129
130     def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
131                          frame_height: float, net: Optional[Net],
132                          court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
133         """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
134
135         A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
136         The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
0.0.
137         """
138         if net is None or not per_frame:
139             return 0.0
140         axis, pos = net

```

```

141     both = 0
142     for _frame_idx, dets in per_frame.items():
143         near, far = False, False
144         for _tid, box in dets:
145             if not person_in_court(box, frame_width, frame_height, court_polygon):
146                 continue
147             fx, fy = _foot_norm(box, frame_width, frame_height)
148             coord = fx if axis == "x" else fy
149             if coord < pos:
150                 near = True
151             else:
152                 far = True
153         if near and far:
154             both += 1
155     return both / len(per_frame)
156
157
158 def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
159                    pad_frac: float) -> bool:
160     """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
161     width/height (the 'adjacent / in-hand' tolerance)?"""
162     x1, y1, x2, y2 = box
163     px = pad_frac * frame_width
164     py = pad_frac * frame_height
165     return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
166
167
168 def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
169                               frame_width: float, frame_height: float,
170                               pad_frac: float = 0.03) -> float:
171     """Fraction of (visible shuttle frames that were also person-sampled) in which the
172     shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
~0**
173     ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
174     """
175     if not per_frame or frame_width <= 0 or frame_height <= 0:
176         return 0.0
177     aligned = 0
178     colocated = 0
179     for p in shuttle_points or []:
180         if not getattr(p, "visible", False):
181             continue
182         dets = per_frame.get(int(p.frame))
183         if dets is None: # shuttle frame wasn't person-sampled ? can't judge
184             continue
185         aligned += 1
186         if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
187               for _tid, box in dets):
188             colocated += 1
189     return colocated / aligned if aligned else 0.0
190
191
192 # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
193 # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
194 PERSON_FEATURE_NAMES = [
195     "players_in_court", "two_sided_motion", "mean_player_motion",
196     "in_court_ratio", "shuttle_player_colocation",
197 ]
198
199
200 def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
201                             frame_width: float, frame_height: float,

```

```

202             court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
203             net: Optional[Net] = None,
204             min_players: int = 2) -> Dict[str, float]:
205     """Aggregate the per-window person features from per-frame detections + the in-
window
206     shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
207     counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
208     return {
209         "players_in_court": players_in_court(counts),
210         "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
211         "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
212         "in_court_ratio": in_court_ratio(counts, min_players),
213         "shuttle_player_colocation": shuttle_player_colocation(
214             per_frame, shuttle_points, frame_width, frame_height),
215     }
216

```

5. assistant (2026-06-19T16:57:13.551Z)

Now let me verify how the gate is wired in the eval harness.

6. user (2026-06-19T16:57:15.087Z)

```

163:         # a normalized court_polygon + optional frame_height + court_height_pad from the
manifest.
164:         for k in ("court_polygon", "frame_height", "court_height_pad"):
190:         # court_polygon, drop candidate rallies whose shuttle is MAJORITY outside the prominent
court ? a
194:         poly = video.get("court_polygon")
196:         from backend.pipeline.detectors.court_gate import gate_intervals_to_court
199:         ivs = gate_intervals_to_court(

```

7. user (2026-06-19T16:57:16.229Z)

```

88-     shuttle net-crossing count (Gate A signal) and ? only when `cue_flags['person']` is
89-     true ? the person-cue features, but it does NOT gate the served handoff unless a
90-     threshold is raised (`min_crossings` / `min_players` default 0 = OFF). The court mask
91-     is optional: `court_polygon=None` ? Gate B counts every detection (player-count-only);
92-     supplied ? off-court persons (spectators / adjacent courts) are excluded.
93-     """
94-     # Which trajectory substrate seeds the windows (shuttle stays primary).
--
107-     # < this. 0 = OFF.
108-     min_players: int = Field(default=0, ge=0)
109-     # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
110-     court_polygon: Optional[list[list[float]]] = None
111-     # Net line for the person two-sided-motion split (person features only ? the shuttle
112-     # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
113-     net_axis: Literal["x", "y"] = "y"

```

8. assistant (2026-06-19T16:57:18.256Z)

Let me read the full wiring context in the eval harness and the FusionConfig context in models.py.

9. user (2026-06-19T16:57:19.692Z)

```

150         with open(manifest_path) as fh:
151             manifest = json.load(fh)
152         out: List[Dict[str, Any]] = []
153         for v in manifest:
154             feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")

```

```

155         gts: List[Interval] = []
156         if os.path.exists(feat):
157             with open(feat) as fh:
158                 gts = [(float(s), float(e)) for s, e in (json.load(fh).get("gts") or
159 [])]
160         entry: Dict[str, Any] = {"name": v["name"], "trajectory": v["trajectory"],
161                                "fps": float(v["fps"]), "frame_width":
162 float(v["frame_width"]),
163                                "gts": gts}
164         # Optional prominent-court gate inputs (multicourt G2; default absent ? the gate
165 is a no-op):
166         # a normalized court_polygon + optional frame_height + court_height_pad from the
167 manifest.
168         for k in ("court_polygon", "frame_height", "court_height_pad"):
169             if k in v:
170                 entry[k] = v[k]
171         out.append(entry)
172     return out
173
174 def windows_for(video: Dict[str, Any], preset: WindowingPreset,
175                 min_crossings: int = 0) -> List[Interval]:
176     """Parse a video's trajectory, window it under ``preset`` ? predicted rally
177 intervals.
178     ``min_crossings`` > 0 applies the **net-crossings rally-state gate** (Gate-A,
179 ``rally_gate.gate_windows``): drop windows whose shuttle crosses the net fewer than
180 this many
181 times (a non-rally fragment). This is the cheapest rally-state cue (W2) ? CPU-only,
182 derived
183 from the trajectory + an auto-estimated net axis; it cleans up the over-split that
184 bounded
185 windowing (W1) leaves behind.
186     NEVER-EMPTY safeguard (mirrors ``FusionSegmenter._select_for_handoff``): if the gate
187 would
188 drop EVERY window of a video that had ?1, fall back to the ungated windows. The gate
189 is
190 strictly *pruning* ? it never zeroes out a non-empty video ? so W2-on-W1 is "do no
191 harm".
192     """
193     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
194     points = TrackNetRunner.parse_trajectory_csv(video["trajectory"])
195     wins = windowing.build_windows(points, video["fps"], video["frame_width"], preset)
196     ivs = windowing.windows_to_intervals(wins)
197     # Prominent-court WINDOW gate (multicourt G2, default-OFF): when the video carries a
198 normalized
199     # court_polygon, drop candidate rallies whose shuttle is MAJORITY outside the
200 prominent court ? a
201     # background/adjacent-court rally. Window-level (not point-level): a background
202 rally's central
203     # tail keeps a few in-court points, enough to hold a window, so point-clearing alone
204 is too leaky
205     # (measured on GX010142 rally #1). No polygon ? no-op. See
206 docs/PROMINENT_COURT_DETECTION.md.
207     poly = video.get("court_polygon")
208     if poly and ivs:
209         from backend.pipeline.detectors.court_gate import gate_intervals_to_court
210         fw = float(video["frame_width"])
211         fh = float(video.get("frame_height") or fw * 9.0 / 16.0)
212         ivs = gate_intervals_to_court(
213             ivs, points, poly, float(video["fps"]), fw, fh,
214             min_in_court_frac=float(video.get("court_min_in_court_frac", 0.5)),
215             height_pad=float(video.get("court_height_pad", 0.0) or 0.0))
216     if min_crossings > 0 and ivs:

```



```

204         from backend.pipeline.detectors.rally_gate import gate_windows
205         gated = [(g.start, g.end)
206                  for g in gate_windows(points, ivs, video["fps"],
min_crossings=min_crossings)
207                  if g.kept]
208         # Gating emptied a non-empty video ? keep the ungated windows.
209         ivs = gated if gated else ivs
210     return ivs
211
212
213     #: One windowed video before scoring: (name, court_type, predicted intervals, GT
intervals).
214     PredictedVideo = Tuple[str, str, List[Interval], List[Interval]]
215
216
217     def predict_all(golden: List[Dict[str, Any]], preset: WindowingPreset,
218                    min_crossings: int = 0) -> List["PredictedVideo"]:
219         """Window every scorable golden video ONCE (the expensive trajectory-parse +
windowing step),

```

10. user (2026-06-19T16:57:20.204Z)

```

80         # so this is purely a speed/disk win; kept opt-in until the side-by-side confirms
it.
81         stream_video: bool = False
82
83
84     class FusionConfig(BaseModel):
85         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
86
87         Every default reproduces control behaviour: the fusion segmenter persists the
88         shuttle net-crossing count (Gate A signal) and ? only when `cue_flags['person']`
is
89         true ? the person-cue features, but it does NOT gate the served handoff unless a
90         threshold is raised (`min_crossings` / `min_players` default 0 = OFF). The court
mask
91         is optional: `court_polygon=None` ? Gate B counts every detection (player-count-
only);
92         supplied ? off-court persons (spectators / adjacent courts) are excluded.
93         """
94         # Which trajectory substrate seeds the windows (shuttle stays primary).
95         substrate: Literal["wasb", "tracknet"] = "wasb"
96         # Windowing velocity threshold for the fusion family (the engine reads
97         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
98         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
99         velocity_threshold: float = 0.02
100         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
101         # enabled ? so the default path never imports torch / touches the GPU.
102         cue_flags: dict[str, bool] = Field(default_factory=dict)
103         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
104         # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
105         min_crossings: int = Field(default=0, ge=0)
106         # Served Gate B: when the person cue is on, drop windows with median in-court
players
107         # < this. 0 = OFF.
108         min_players: int = Field(default=0, ge=0)
109         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
110         court_polygon: Optional[list[list[float]]] = None
111         # Net line for the person two-sided-motion split (person features only ? the shuttle
112         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
113         net_axis: Literal["x", "y"] = "y"
114         net_position: float = Field(default=0.5, ge=0.0, le=1.0)

```

```

115
116
117     class IndexingConfig(BaseModel):
118         default_segmenter: str = "yolo_hybrid"
119         use_gpu: bool = True

```

11. assistant (2026-06-19T16:57:24.818Z)

I have everything needed. Let me confirm one detail about the "single tracked trajectory" claim ? that the gate operates on the single WASB trajectory, not multiple tracks. The docstring and wiring confirm `points = TrackNetRunner.parse\_trajectory\_csv(...)` is the single tracked trajectory. Let me verify the trajectory parse returns one trajectory.

12. user (2026-06-19T16:57:26.478Z)

```

237:     def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
238-         """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
239-         points: List[TrajectoryPoint] = []
240-         with open(csv_win_path, newline="") as f:
241-             reader = csv.DictReader(f)
242-             for row in reader:
243-                 try:
244-                     points.append(TrajectoryPoint(
245-                         frame=int(row["Frame"]),
246-                         visible=int(row["Visibility"]) == 1,
247-                         x=float(row["X"]),
248-                         y=float(row["Y"]),
249-                     ))

```

13. assistant (2026-06-19T16:57:30.158Z)

Confirmed. `parse\_trajectory\_csv` returns a single flat `List[TrajectoryPoint]` (one (x,y,visible) per frame ? a single tracked trajectory, no per-track separation). All claims verified.

14. user (2026-06-19T16:57:46.950Z)

Structured output provided successfully

15. assistant (2026-06-19T16:57:54.742Z)

Verification complete. All claims checked against the actual source.

Summary of verdicts:

- gate_intervals_to_court is window-level: VERIFIED (`court\_gate.py:61-78`)
- Subtractive-only (drop, never recover): VERIFIED (`court\_gate.py:74-78`, docstrings lines 11/73)
- Operates on the single tracked trajectory: VERIFIED (`rally\_seg\_eval.py:186,200`; `tracknet\_runner.py:237-249`)
- point_in_polygon exists + reusable as selection term: VERIFIED (`fusion\_features.py:30-48`, reused at `court\_gate.py:25,36-38`)
- FusionConfig.court_polygon exists + reusable: VERIFIED (`models.py:84,109-110`)
- Wired into production: REFUTED ? the only call site is the offline eval harness (`backend/eval/rally\_seg\_eval.py:196-202`, gated on a manifest `court\_polygon`, default-OFF). `FusionConfig.court\_polygon` feeds the person-cue Gate B path, not `gate\_intervals\_to\_court`.

Relevant file paths:

- C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\court_gate.py
- C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\fusion_features.py
- C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py
- C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\rally_seg_eval.py
- C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\tracknet_runner.py